Stroke Risk Prediction Through Supervised Machine Learning Models

Filippo Avon[1], Irida Pajenga[1], Carmelo Ferraro[1], Lorenzo Luciano Gulizia[1]

Abstract
The stroke represents one of the most prominent causes of death and disability in global level, making its prior identification fundamental for the individuals in risk. This work treats the possibility to predict the onset of a stroke based on clinical, demographic, and behavioral variables. The analysis focuses on the proper identification of the most probable risk factors, taking into consideration the gender, age, hypertension, present heart disease, marriage status, work type, average glucose levels, body mass index and the residence location of the patient in examination. Having these variables, the point of interest is to apply supervised classification models to estimate the probability of stroke and evaluate their predictive ability, contributing to the early identification of the most vulnerable patients.

Keywords
Stroke — Prediction — Machine Learning — Supervised Learning

[1] *Department of Computer Science, University of Milano-Bicocca, Milan, Italy*

Contents

Introduction 1
1 Description Dataset 1
1.1 Variables . . . 2
2 Preprocessing 2
2.1 Dealing with missing data for Training . . . 2
2.2 Removal of non-informative and redundant attributes 3
3 Model Training 3
3.1 Description of the training set . . . 3
3.2 Cross-Validation and Balancing Strategy . . . 3
3.3 Algorithm Configuration . . . 4
4 Model Validation 4
4.1 Description of the test set . . . 4
4.2 Validation Workflow Architecture . . . 4
4.3 Dynamic Thresholding Strategy (Grid Search) . . . 4
4.4 Evaluation Metrics . . . 4
5 Competitive Models 4
5.1 Probabilistic models . . . 5
5.2 Tree-based models . . . 5
5.3 Ensemble methods . . . 5
5.4 Artificial neural networks . . . 5
5.5 Cross Validation with Oversampling . . . 5
Comparative Discussion
5.6 ROC Curve . . . 6
6 Choice of the threshold 6
7 Score and results 6
References 7

Introduction

Numerous studies have highlighted how the onset of a stroke is influenced by variables such as age, the presence of previous pathologies, lifestyle, and certain biological parameters. The joint analysis of this information requires tools capable of managing heterogeneous data and identifying complex relationships between variables, often not immediately observable using traditional statistical approaches. In this context, machine learning techniques represent a valuable support for healthcare data analysis, enabling the construction of predictive models capable of estimating the risk of future clinical events. Specifically, supervised classification models allow learning from historical data and providing predictions based on recurring patterns within the analyzed population. This work is based on a healthcare dataset that collects demographic, clinical, and behavioral information from patients, with the aim of analyzing stroke risk. Through the application of machine learning techniques, the study aims to identify the main risk factors associated with the event and evaluate the effectiveness of predictive models in the context under consideration.

1. Description Dataset

According to the World Health Organization (WHO) [1] stroke is the 2nd leading cause of death globally, responsible for approximately 11% of total deaths. This dataset is used to

predict whether a patient is likely to get stroke based on the input parameters like gender, age, various diseases, and smoking status, and is composed of 5112 records and 12 attributes. Each row in the data provides relevant information about the patient.

In the analyzed dataset [2], stroke cases represent a very small percentage compared to negative cases (about 10%), with a total number of events equal to approximately 150. This condition makes machine learning particularly complex, since a model trained without adequate preprocessing techniques tends to favor the majority class, obtaining high apparent accuracy but poor predictive capacity for clinically relevant cases.

1.1 Variables

In particular, the following covariates were detected for each patient.

- **id**: unique identifier.
- **gender**: Male, Female, or Other.
- **age**: age of the patient.
- **hypertension**: 0 if the patient does not have hypertension, 1 otherwise.
- **heart_disease**: 0 if the patient does not have heart disease, 1 otherwise.
- **ever_married**: No or Yes.
- **work_type**: children, Govt_job, Never_worked, Private, or Self-employed.
- **Residence_type**: Rural or Urban.
- **avg_glucose_level**: average glucose level in blood.
- **bmi**: body mass index.
- **smoking_status**: formerly smoked, never smoked, smokes, or Unknown.
- **stroke**: 1 if the patient had a stroke, 0 otherwise.

Note: Unknown in **smoking_status** indicates missing information for the patient. In Figure 1 and Figure 2, some of the variables statistics are illustrated.

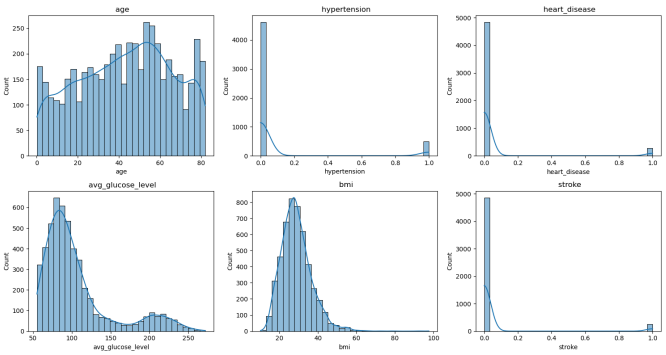


Figure 1. Data about patients

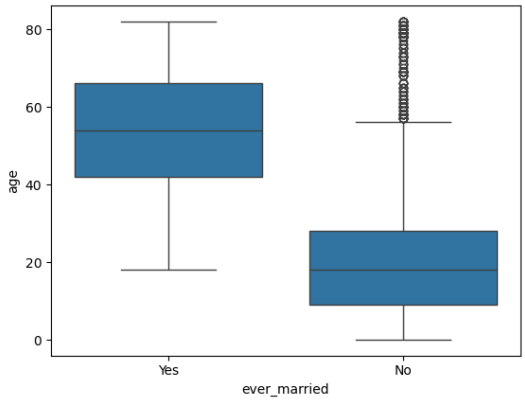


Figure 2. Marital status

2. Preprocessing

Prior to model training, the dataset underwent a preprocessing pipeline aimed at improving data quality and ensuring compatibility with the learning algorithms. Records belonging to the `gender = Other` category were removed prior to preprocessing. Binary categorical variables (`gender`, `ever_married`, `Residence_type`) were encoded as 0/1. Nominal variables with more than two levels (`work_type`, `smoking_status`) were transformed using one-hot encoding. One-hot encoding is a categorical encoding technique that converts a nominal variable with k categories into k binary (0/1) indicator variables.

Following encoding, raw categorical attributes and the identifier column were discarded, retaining only numeric and encoded features. The dataset was subsequently partitioned using stratified sampling on the target variable `stroke`, generating a 60% training set and a 40% hold-out set with fixed random seed (1678807467440). The hold-out set was then split using a relative 75/25 stratified partition, resulting in a 30% validation set and a 10% test set.

2.1 Dealing with missing data for Training

An initial exploratory analysis revealed a significant percentage of missing values, 201 records (3.93%) that contain at least one missing value. Missingness is exclusively confined to the `bmi` attribute. Given the already limited number of stroke cases, eliminating incomplete rows would have resulted in an unacceptable loss of information.

For this reason, we chose not to delete any records, but to manage the missing values through an imputation phase. Missing data manipulation was performed using an R snippet.

Before addressing class imbalance and data partitioning, missing values in `bmi` were handled through multiple imputa-

tion using the `mice` package. Prior to imputation, categorical variables were cast to factors and numeric variables to numeric types to ensure compatibility with MICE. BMI was imputed using Predictive Mean Matching (PMM), with predictors restricted to demographic and clinical covariates (`gender`, `age`, `hypertension`, `heart_disease`, `ever_married`, `work_type`, `Residence_type`, `avg_glucose_level`, `smoking_status`) and the missingness indicator `bmi_na`. The target variable `stroke` was explicitly excluded from the imputation model to prevent information leakage. The procedure was run with $m = 5$ imputations and 10 iterations, and the completed dataset was obtained from the first imputed version.

2.2 Removal of non-informative and redundant attributes

During the preprocessing phase, particular attention was paid to the identification and removal of non-informative (superfluous) attributes, i.e. variables that do not contribute useful information to the classification task.

First, an analysis of feature variance was performed in order to detect variables with null or very low variance. Such attributes were removed, as they are unable to discriminate between classes and may introduce unnecessary noise into the learning process.

In addition, categorical variables that, after preprocessing, exhibited only a single level were excluded from the dataset. Since these variables do not provide variability across instances, they are not informative for predictive modeling.

This phase also considered the issue of redundancy among variables. Highly correlated features can negatively affect model stability and interpretability, especially in the presence of a limited number of observations. For this reason, the potential presence of multicollinearity was taken into account as an important aspect to be analyzed and controlled.

3. Model Training

The training phase focused on the development and comparative analysis of supervised classification models to predict stroke risk. To ensure a comprehensive evaluation of different learning paradigms, seven competitive algorithms were selected, ranging from probabilistic models to ensemble methods and neural networks: Logistic Regression, Naive Bayes, Decision Tree (J48), Random Forest, AdaBoost, LogitBoost, and Multilayer Perceptron (MLP).

3.1 Description of the training set

The dataset preparation followed a rigorous multi-stage balancing strategy. Starting from the original highly imbalanced distribution (approximately 95% negative and 5% positive cases), an initial **undersampling** phase was applied to reduce the majority class, bringing the ratio to approximately **90% negative and 10% positive**.

To guarantee an unbiased evaluation and prevent overfitting, this pre-balanced dataset was then partitioned into three independent subsets using a **stratified sampling** approach based on the target variable `stroke`. The splitting process followed a hierarchical structure:

- **Training Set (60%):** This subset represents the core of the learning process. It is within this partition that the models are trained.
- **Validation Set (30%):** Derived from the remaining data, this set is used exclusively for **model comparison** and **threshold optimization**. Unlike standard approaches, it was not used for hyperparameter tuning, as the models were evaluated using robust baseline configurations (or specifically selected parameters as in the case of Random Forest).
- **Test Set (10%):** The final partition acts as a strictly hold-out set, reserved for the conclusive assessment of the selected best model.

To ensure the reproducibility of the experiment, a fixed random seed (1678807467440) was applied to all partitioning nodes.

3.2 Cross-Validation and Balancing Strategy

The reliability of the results was maximized by implementing a **10-fold Cross-Validation** scheme. Instead of a single train-test split, the training set was divided into 10 folds, ensuring that the performance metrics reflect the model's stability.

A critical aspect of the training methodology was the handling of class imbalance within the cross-validation loop. Although the input data had a prevalence of $\approx 10\%$ stroke cases (due to the prior undersampling), this was still suboptimal for learning complex patterns. Therefore, the **SMOTE** (Synthetic Minority Over-sampling Technique) [3] algorithm was integrated directly into the training process using a *FilteredClassifier*. The SMOTE filter was configured with the following hyperparameters:

- **Nearest Neighbors ($K = 5$):** Synthetic instances were generated by interpolating between the 5 nearest neighbors.
- **Oversampling Percentage ($P = 100.0\%$):** The minority class was effectively doubled within the training folds. This transformation shifted the class distribution from the input $\approx 90/10$ ratio to a more balanced ratio of approximately **80% No-Stroke and 20% Stroke**.

By encapsulating SMOTE within the *FilteredClassifier*, the oversampling was applied **exclusively to the training data** of each fold, strictly preventing data leakage.

3.3 Algorithm Configuration

To establish a robust baseline, most algorithms were trained using their default hyperparameters within the KNIME Analytics Platform (Weka 3.6 implementation). An exception was made for the **Random Forest** classifier, which was explicitly configured with **500 iterations** (trees) instead of the default value, to ensure convergence and maximize the stability of the ensemble predictions.

4. Model Validation

The validation strategy represents a critical component of the predictive pipeline. Given the clinical nature of the problem, relying on standard metrics provided by default classifiers (which typically operate at a static 0.5 threshold) would be insufficient. Therefore, a custom validation workflow was designed to evaluate the models' performance under dynamic conditions, specifically implementing a **Threshold Optimization** loop.

4.1 Description of the test set

The integrity of the final evaluation relies on a rigorous separation of data. The **Test Set** consists of the final **10%** of the dataset. Crucially, this subset generated via stratified sampling has remained **inaccessible ("blind")** to the models throughout the entire training and optimization pipeline. It serves as an unbiased benchmark for the final results presented in section 7.

4.2 Validation Workflow Architecture

To perform a granular analysis of model sensitivity, a dedicated validation branch was implemented in KNIME. This workflow processes the validation data through a specific sequence of operations:

1. **Preprocessing for Validation:** Before entering the evaluation loop, the validation data undergoes a specific transformation pipeline implemented via *Math Formula* nodes to ensure consistency with the training data.
2. **Probability Generation:** The trained models are applied to the validation set using *Weka Predictor (3.6)* nodes. Crucially, these nodes are configured to output the **class probabilities**. The specific probability of the positive class ($P(\text{Stroke} = 1)$) is extracted and serves two fundamental purposes:
    - It is the direct input for computing the **AUC** (Area Under the Curve).
    - It acts as the variable against which the dynamic threshold is compared in the subsequent optimization step.

4.3 Dynamic Thresholding Strategy (Grid Search)

A defining feature of this study is the implementation of a **Counting Loop** to perform a Grid Search for the optimal classification threshold based on the predicted probabilities.

- **Step 1: Dynamic Threshold Calculation.** Inside the loop, a *Math Formula (Variable)* node defines the testing threshold (T) for the current iteration using the linear equation:

  $$T_i = 0.06 + (i \times 0.005)$$

  This scans the sensitivity range starting from a probability of 0.06.
- **Step 2: Custom Classification Rule.** A *Rule Engine* node compares the probability output by the Weka Predictor against the current threshold. A patient is classified as positive (Stroke) only if:

  $$P(\text{Stroke} = 1) \geq T_i$$

- **Step 3: Performance Collection.** At each iteration, a *Scorer* node computes the confusion matrix. The resulting metrics (Recall, Precision) are collected via a *Loop End* node, generating a table that maps every probability threshold to its corresponding clinical performance.

4.4 Evaluation Metrics

The outputs of the custom loop were used to compute the following key metrics, prioritizing clinical safety:

- **Sensitivity (Recall):** The primary metric for this study. Maximizing Recall (1) is the priority to prevent missed diagnoses.

  $$\text{Recall} = \frac{TP}{TP + FN} \tag{1}$$

- **Precision (2):** Used to monitor the rate of False Positives.

  $$\text{Precision} = \frac{TP}{TP + FP} \tag{2}$$

- **F-Measure (3):** Used to identify the threshold that offers the best balance.

  $$\text{F-measure} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{3}$$

- **AUC (Area Under the Curve) (4):** Calculated directly from the probability scores, providing a threshold- independent measure of discriminative power.

  $$\text{AUC} = P\big(s(x^+) > s(x^-)\big) \tag{4}$$

5. Competitive Models

Indeed, it has been found that using classification models, both parametric and non-parametric, based on unbalanced datasets leads to high *Accuracy* for the most frequent events and low *Accuracy* for rare events. Accuracy is given by the formula:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{5}$$

where:

- TP and TN represent the number of correctly classified instances belonging to the positive and negative classes respectively

- FP and FN indicate the number of instances (negative and positive) misclassified.

5.1 Probabilistic models

Among the most widely used methods and that we applied are those based on probabilistic assumptions such as the **Naive Bayes** classifier (6) and **Logistic regression**, which allow, starting from the large amount of data, to obtain accurate results under the assumption of independence between attributes.

$$p(c_j \mid x) = \frac{\prod_i p(x_i \mid c_j)\, p(c_j)}{p(x)} \tag{6}$$

5.2 Tree-based models

Decision Tree J48 classifier. Regression trees were not considered, as the prediction task is binary classification. J48 (C4.5 in WEKA) builds an univariate decision tree, i.e., it performs splitting using one attribute at each internal node [4]. Some basic steps are given below to construct tree:

- First, check whether all cases belongs to same class, then the tree is a leaf and is labeled with that class.

- For each attribute, calculate the information and information gain.

- Find the best splitting attribute (depending upon current selection criterion) [5]

Quinlan [6] also covers a number of limitations of C4.5, such as its bias in favor of rectangular regions, and he discusses some ideas for extending the abilities of the algorithm.

5.3 Ensemble methods

AdaBoost, **LogitBoost**, and **Random Forest** classifier that is an ensemble of decision trees.

Adaboost seeks to improve classification performance by minimizing the error rate. During each iteration, a tree is estimated on the dataset and predictions are calculated. Each subject is then assigned a weight that increases in the next iteration if it is incorrectly classified. In the next iteration, a sample is constructed by extracting subjects with the highest weight, and the algorithm will then work on those subjects. The tuning parameter is the number of iterations.

LogitBoost was selected as a boosting algorithm that minimizes logistic loss, making it well suited for probabilistic classification and improving performance on difficult-to-classify instances.

A random forest is an ensemble model that combines several trees obtained from different bootstrap samples of the data. For each sample, a subset of k random covariates is selected. This approach reduces the variability of individual trees. The tuning parameters in a random forest are the number of covariates randomly selected from each tree and the number of bootstrap samples.

5.4 Artificial neural networks

These are based on a collection of interconnected units or nodes called artificial neurons, which, once they have received and processed the signal, send it to the neurons connected to them. Among the classifiers belonging to this category of algorithms, the **Multilayer Perceptron** has been used, which uses the backward error propagation technique to classify instances.

5.5 Cross Validation with Oversampling

Table 1 reports the performance of the considered classifiers evaluated through cross-validation after applying oversampling techniques to mitigate class imbalance. Overall, the results show that oversampling enables all models to better recognize the minority class, as reflected by non-zero Recall values across classifiers. However, the improvement in Recall is generally accompanied by a reduction in Precision, confirming the typical trade-off observed in imbalanced classification problems.

Table 1. Performance comparison of classification models using Cross validation with Oversampling

Model	Recall	Precision	F-measure	AUC
Logistic	0.396	0.434	0.414	0.852
LogitBoost	0.255	0.521	0.342	0.848
Random Forest	0.215	0.427	0.286	0.825
Naive Bayes	0.631	0.325	0.429	0.823
AdaBoost	0.315	0.461	0.375	0.802
J48	0.295	0.411	0.344	0.801
MLP	0.376	0.392	0.384	0.769

In particular, Naive Bayes achieves the highest Recall (0.631), indicating a strong ability to detect stroke cases. This behavior, however, comes at the expense of Precision (0.325), suggesting a substantial number of false positives. A similar pattern can be observed for MLP, which attains a relatively high Recall (0.376) but moderate Precision, reflecting a tendency to over-predict the positive class.

Logistic Regression and MLP provide a more balanced trade-off between Recall and Precision, resulting in competitive F-measure values (0.414 and 0.384 respectively). Moreover, Logistic Regression records the highest AUC (0.852), indicating strong discriminative capability independently of the decision threshold.

Ensemble methods such as AdaBoost, Random Forest, and J48 show moderate Recall values and comparatively lower F-measure scores, suggesting that oversampling alone is not

sufficient for these models to effectively capture the minority class patterns.

5.5.1 Comparative Discussion

It emerges that oversampling primarily impacts Recall, improving the detection of stroke cases for most classifiers, while Precision tends to decrease, due to the increased number of false positives.

5.6 ROC Curve

The ROC curve was used as a metric to choose the model that performs best in training, and then take it to validation.

ROC analysis was performed to assess the discriminative capability of the evaluated classifiers under different class balancing strategies. Model performance was assessed using cross-validation, both on the original imbalanced dataset and on a balanced version obtained through oversampling applied within the training folds.

Among all considered models, Logistic Regression consistently achieved the highest Area Under the Curve AUC = 0.852 (Figure 3, indicating a stable and robust discriminative performance across different data distributions.

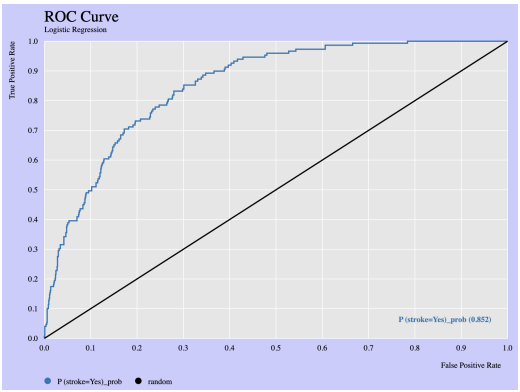


Figure 3. Logistic Regression ROC

In addition to its superior AUC, Logistic Regression was selected as the final model due to its simplicity, stability, and interpretability, which are particularly important in a clinical context. Unlike more complex ensemble or neural models, Logistic Regression provides transparent decision boundaries and allows for a direct interpretation of feature contributions, facilitating clinical trust and practical deployment.

Given its consistent performance, robustness to class imbalance, and strong discriminative capability, the Logistic Regression model was chosen for reporting and further analysis, and its corresponding ROC curve is presented.

6. Choice of the threshold

We chose the threshold on the validation set and on data that has not been processed and on which there is no oversampling, in other words we selected the pure original dataset.

The selection of the decision threshold was not performed using a single optimal value, but through a constraint-based procedure driven by clinical considerations. In the context of stroke prediction, failing to identify a positive case represents a critical error; therefore, the threshold selection was guided by minimum Recall requirements. Two Recall constraints were considered: Recall ≥ 0.80 and Recall ≥ 0.85. These values represent increasingly conservative operating conditions, ensuring a high sensitivity to stroke cases while allowing the analysis of the corresponding trade-offs in Precision and F-measure. All model configurations that did not satisfy the imposed Recall constraints were discarded.

The observed reduction in Precision is a direct consequence of enforcing high Recall constraints. In the context of stroke prediction, false negatives represent a critical risk, as they correspond to patients whose condition is not identified. For this reason, the threshold selection strategy **prioritizes** sensitivity over specificity, accepting a higher number of false positives in order to minimize missed stroke cases.

In Table 2 we rank by the number of false positives, even though the main point of interest are the false negatives, because for the latter, previously we already found the best models. Logistic Regression achieves the lowest number of false positives at a threshold of 0.14 and was therefore selected for the final model.

7. Score and results

Before the final evaluation, the test set was processed using the same preprocessing pipeline adopted during training. All transformations, including feature selection, scaling and data cleaning, were applied using the parameters learned exclusively from the training phase. This step ensures a fair and unbiased evaluation of the final model, preventing any form of information leakage from the test set into the training process.

Analysis of performance metrics shows that the best classifier of the four is Logistic Regression. In fact, it has the highest precision, recall (very important in the medical field), and AUC of the corresponding ROC curve.

Table 3. Confusion matrix for Logistic Regression on the test set with threshold $t = 0.14$.

	Predicted Yes	**Predicted No**
Actual Yes	22	3
Actual No	148	338

Furthermore, the final evaluation was performed using Logistic Regression. The classifier was applied to the independent test set using the same preprocessing pipeline adopted during training, ensuring a fair and unbiased assessment.

The model outputs posterior class probabilities, which were converted into binary predictions through an explicit decision threshold applied to $P(\text{stroke} = \text{Yes})$. Two operating thresholds were evaluated, namely $t = 0.14$ and $t = 0.17$.

Table 2. Top model configurations satisfying Recall ≥ 0.85, ranked by the number of false positives.

Model	Threshold	Recall	False Positives	False Negatives
Logistic Regression	0.14	0.853	481	11
Naive Bayes	0.135	0.853	483	11
Logistic Regression	0.135	0.853	485	11

At threshold $t = 0.14$ in Table 3, the model correctly identifies 22 out of 25 stroke cases, achieving a Recall of 0.88. However, this configuration produces a high number of false positives, reflecting a highly permissive decision policy.

Increasing the threshold to $t = 0.17$ preserves the same Recall level while reducing the number of false positives from 148 to 131, as shown in Tables 3 and 4. This leads to an improvement in overall accuracy and represents a more favorable trade-off between sensitivity and specificity.

Table 4. Confusion matrix for Logistic Regression on the test set with threshold $t = 0.17$.

	Predicted Yes	Predicted No
Actual Yes	22	3
Actual No	131	355

Based on these results, the threshold $t = 0.17$ in Table 4 was selected as the final operating point for the Logistic Regression model.

References

[1] World Health Organization. Stroke. https://www.who.int/news-room/fact-sheets/detail/stroke, 2023. Accessed: January 2026.

[2] Federico Soriano. Stroke prediction dataset. https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset, 2021. Accessed: February 2026.

[3] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. Smote: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.

[4] G. Manikanta, M. D. Adul Gafar, and S. Govinda Raju. Decision tree analysis on j48 algorithm for data mining. *Journal of Science and Technology*, 7(5):200–205, July 2022.

[5] Thales S. Korting. C4.5 algorithm and multivariate decision trees. Web, 2013. Accessed: February 2, 2026.

[6] J. R. Quinlan. *C4.5: Programs for Machine Learning*. Morgan Kaufmann, San Mateo, CA, 1992.